UNIVERSIDAD DEL VALLE DE GUATEMALA

CONSTRUCCIÓN DE COMPILADORES

Pablo



Fase 3 Generación de código MIPS

Ihan Marroquin - 23108

Diego Patzan - 23525

GUATEMALA, 08 de noviembre del 2026

Informe final

https://github.com/UVG-Tareas-y-proyectos/fase_compilacion_compis

Arquitectura de la generación de código

Extendimos Compiscript con un backend que traduce el código de tres direcciones de la fase 2 a ensamblador MIPS32. El archivo generado se ensambla y ejecuta en MARS. Conservamos el frontend YALex y YAPar, el análisis semántico, la tabla de símbolos y el generador de TAC del proyecto anterior. Las expresiones, llamadas y estructuras de control continúan pasando por las mismas validaciones antes de generar instrucciones destino.

```
Compiscript -> árbol -> semántica -> TAC -> MIPS32 -> MARS
```

MIPSGenerator recibe el TAC validado y recorre los cuádruplos por procedimiento. Cada operación produce instrucciones MIPS o una llamada a una rutina MIPS de apoyo. El archivo exportado incluye datos, procedimientos y runtime. El compilador no ejecuta Compiscript para construir la salida, ni coloca una tabla de cuádruplos para interpretarla dentro del simulador.

Componente	**Responsabilidad**
pipeline.py	Coordinar las tres fases y recoger diagnósticos.
mips_model.py	Calcular marcos físicos, asignar registros y aplicar mirilla.
mips.py	Traducir los operadores, etiquetas, llamadas y retornos TAC.
mips_runtime.asm	Implementar valores, entornos, listas, objetos y errores en MIPS.
mips_runner.py	Invocar MARS y recoger salida, errores y código de terminación.
scripts/compiscript.py	Compilar, exportar y ejecutar desde la terminal.
scripts/ide.py	Editar, revisar las representaciones y ejecutar desde el IDE.

La selección de instrucciones y la asignación de registros siguen los conceptos de Generación de código. Los marcos, enlaces y secuencias de llamada corresponden a Entornos en tiempo de ejecución y Asignación de memoria para procedimientos. La optimización final aplica una mirilla conservadora. La salida principal es un archivo .asm para la arquitectura educativa MIPS32.

Convención de llamadas y retornos

Definimos una convención interna para funciones y métodos. El llamador evalúa los argumentos y guarda sus referencias en un área de su marco. Pasa la closure en $a0, la dirección del vector de argumentos en $a1 y la cantidad en $a2. La función devuelve una referencia en $v0. Esta convención permite más de cuatro parámetros y se utiliza de forma uniforme en todo el código generado; es independiente de la ABI o32 de C.

La closure contiene la dirección del procedimiento, su entorno léxico de definición y el receptor cuando es un método. El procedimiento comprueba la cantidad de argumentos, reserva su marco moviendo $sp y conserva el estado del llamador. El retorno restaura los registros y la pila antes de ejecutar jr $ra. Las llamadas indirectas utilizan jalr.

Desplazamiento	**Contenido del marco MIPS32**
0	Dirección de retorno $ra.
4	$fp anterior, enlace de control.
8 y 12	$s0 y $s1 anteriores: entorno y closure.
16	Manejador de errores que existía al entrar.
20	Dirección del vector de argumentos de entrada.
24 y 28	$s2 y $s3 anteriores.
Desde 32	Hogares de temporales y área de argumentos para otras llamadas.

Cada marco se alinea a ocho bytes. $fp conserva una base estable aunque rutinas de apoyo utilicen temporalmente la pila. Cada llamada recursiva obtiene un marco independiente, de manera que los temporales y argumentos de una activación no reemplazan a los de otra. El límite de profundidad es de 256 activaciones.

El diseño abstracto de fase 2 utilizaba palabras de ocho bytes. El backend calcula un diseño físico distinto para MIPS32, donde cada referencia ocupa cuatro bytes. Los locales y parámetros se resuelven mediante celdas de sus entornos en el heap; el marco guarda los enlaces y las referencias temporales necesarias. El enlace de acceso proviene del entorno capturado por la closure, mientras que el enlace de control permite recuperar al llamador.

Asignación de registros y selección de instrucciones

RegisterAllocator mantiene un descriptor que relaciona cada registro temporal con el valor TAC que contiene, y una marca para valores modificados. getReg busca primero el valor ya disponible; si no existe, toma un registro libre. Cuando todos están ocupados, guarda el menos recientemente utilizado en su hogar de pila y reutiliza ese registro. Una lectura posterior carga el valor desde su desplazamiento respecto de $fp.

Utilizamos $t0 a $t7 para temporales y reservamos $t8 y $t9 como apoyo del generador. Los registros $s conservan contexto y se salvan al entrar al procedimiento. Antes de llamadas, saltos y etiquetas vaciamos los descriptores para que cada camino encuentre los valores correctos en memoria. La política prioriza corrección y claridad sobre una asignación global de registros.

Operación TAC	**Traducción destino**
copy y declare	Cargar o conservar una referencia; declarar una celda del entorno.
+, -, *, / y %	Llamar a rutinas MIPS con instrucciones enteras o de punto flotante.
Comparación	Producir un boolean mediante comparaciones MIPS.
goto, if e ifFalse	Emitir etiquetas, saltos y ramas condicionales.
param, call y return	Guardar argumentos, llamar con jalr y restaurar el marco al retornar.
Acceso a lista	Comprobar límites y calcular la dirección del elemento.
Campos y métodos	Localizar el campo o buscar el método en la jerarquía de clases.

Los valores se representan mediante referencias a registros con etiqueta de tipo. Por ejemplo, la suma recibe las referencias de sus operandos y utiliza addu para enteros o add.d para double. Esta representación permite que las funciones y closures intercambien valores sin utilizar objetos de Python durante la ejecución.

Optimización por mirilla

La pasada local elimina movimientos de un registro hacia sí mismo, cargas consecutivas idénticas y saltos a la etiqueta inmediatamente siguiente. Conserva las llamadas y los NOP de los demás saltos. Puede desactivarse con --sin-mirilla. Las pruebas ejecutan las dos versiones y comparan su salida; no se implementa un optimizador global ni un GDA de subexpresiones.

Valores y memoria en tiempo de ejecución

Los datos constantes se ubican en la sección .data y las instrucciones en .text. La pila conserva los marcos de llamada; el heap contiene valores y entornos que pueden sobrevivir al retorno. Las rutinas reservan bloques alineados mediante el servicio sbrk de MARS. Estas reservas permanecen hasta finalizar la simulación; no implementamos recolección de basura para el heap MIPS.

Etiqueta	**Representación**
null y boolean	Valores constantes compartidos.
integer	Entero con signo de 32 bits.
float	Double alineado dentro del registro del valor.
string	Referencia a bytes Latin-1 terminados en NUL.
lista	Longitud y referencia al vector de elementos.
closure	Código, entorno de definición y receptor.
clase e instancia	Descriptor, clase padre, entorno y campos.

Los entornos enlazados permiten buscar una variable en su ámbito y después en sus padres. Una closure conserva el entorno de definición aunque la función que lo creó haya terminado. Las instancias mantienen campos independientes. La búsqueda de métodos comienza en la clase hija y continúa hacia los padres; los inicializadores de los campos se ejecutan primero para el padre y después para la hija.

La división entre integer trunca hacia cero y el residuo conserva el signo del dividendo. Se detectan desbordamientos de operaciones de enteros y se rechazan literales fuera del rango MIPS32. Los float utilizan double y se rechazan resultados no finitos. El formato de impresión corresponde a MARS, por lo que su notación científica puede diferir de la utilizada por Python.

Las cadenas admiten los caracteres de Latin-1, incluidos los acentos del español. Un carácter fuera de ese conjunto o un NUL interno produce un diagnóstico de generación. Las listas, objetos y closures se comparan por identidad. Estos criterios forman parte de la representación física elegida para el backend.

Errores y recuperación

Cada instrucción conserva su ubicación de origen. El runtime informa división entre cero, índices fuera de rango, desbordamientos y profundidad de llamadas. try/catch registra un punto de recuperación y el estado de pila y entorno. Si el error nace dentro de otra función, se recupera el manejador exterior. Un error no capturado muestra línea y columna y termina con código distinto de cero.

Instalación y ejecución del compilador

Se requiere Python 3.10 o posterior con Tkinter. Desde el repositorio se crea el entorno, se instalan las dependencias y se prepara MARS. El instalador descarga el JAR oficial, verifica su SHA-256 y prepara Java 17 portátil si no encuentra Java en PATH. Las herramientas se guardan en tools y no se incluyen en Git. La preparación utiliza Internet una vez; las ejecuciones posteriores pueden realizarse sin conexión.

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe scripts\preparar_mips.py
.\.venv\Scripts\python.exe scripts\ide.py
```

Compilar o F5 genera las representaciones y permite revisar Árbol, Símbolos, Errores, TAC, Memoria y MIPS. Ejecutar MIPS o F6 invoca el simulador en segundo plano y coloca la salida en Salida MIPS. Exportar MIPS guarda el ensamblador completo. Ejecutar TAC y Exportar TAC conservan las herramientas anteriores. Una nueva compilación limpia las salidas y descarta resultados pendientes de una ejecución anterior.

Para generar y ejecutar desde la terminal:

```
.\.venv\Scripts\python.exe scripts\compiscript.py `
  examples\fase3_llamadas.cps --salida-mips programa.asm
.\.venv\Scripts\python.exe scripts\compiscript.py `
  examples\fase3_llamadas.cps --ejecutar-mips
.\.venv\Scripts\python.exe scripts\compiscript.py `
  examples\basico.cps --mips --mapa-mips
```

--mips muestra el ensamblador, --mapa-mips muestra los marcos físicos y --sin-mirilla desactiva la optimización local. --salida programa.asm también exporta MIPS; .tac y .json conservan su comportamiento anterior. --tiempo-mips limita segundos y --limite-mips instrucciones simuladas. Los valores por defecto son 15 segundos y 2 000 000 instrucciones MIPS.

El archivo .asm puede abrirse en MARS y ejecutarse mediante Run, Assemble y Go. También puede utilizarse directamente el comando siguiente; si Java está en PATH se reemplaza la primera ruta por java.

```
.\tools\jre\bin\java.exe -jar tools\Mars4_5.jar nc me sm programa.asm
```

Para otra instalación se configuran MARS_JAR y COMPISCRIPT_JAVA. La CLI devuelve 0 al completar correctamente, 1 ante errores de compilación o ejecución y 2 ante errores de lectura o escritura. El backend incluye NOP después de saltos y llamadas, y se prueba con delayed branching activado y desactivado.

Validación y ejemplos de la entrega

La batería completa contiene 182 pruebas aprobadas en el entorno de entrega. Conservamos las pruebas del frontend, semántica y TAC, y agregamos comprobaciones de los marcos físicos, derrames de registros, generación y ejecución en MARS. Los casos de comportamiento anteriores se ejecutan ahora como ensamblador y se comparan con sus resultados esperados. También contrastamos salidas con el intérprete de TAC.

Caso	**Resultado comprobado**
fase3_llamadas.cps	720, 15 y 25: recursión, cinco parámetros y conservación de valores.
fase3_memoria.cps	11, 12, 21 y 24: closures independientes, herencia y listas.
fase3_control.cps	8, 3, 3.5, false, error de índice capturado y continuación.
fase3_error_generacion.cps	Diagnóstico de literal fuera del rango MIPS32.
fase3_error_ejecucion.cps	División entre cero reportada al ejecutar MARS.
Ensamblador inválido	El simulador rechaza una instrucción inexistente.
Ciclo infinito	El límite de instrucciones produce un resultado fallido.
IDE y CLI	Generación, exportación, ejecución y limpieza de resultados anteriores.

Se comprueban llamadas anidadas y recursión mutua, métodos ligados a un objeto, listas anidadas, cortocircuito, valores capturados después de retornar, errores recuperados desde funciones internas y operaciones numéricas. Las pruebas con mirilla activada y desactivada verifican que los resultados se conserven.

```
.\.venv\Scripts\python.exe -m pytest -q
```

Las pruebas que dependen de MARS requieren completar la preparación del simulador. Si Java o el JAR no están disponibles, pytest las identifica como omitidas. El flujo de GitHub Actions instala Python y Java, prepara MARS y ejecuta la batería. El IDE puede compilar y exportar sin Java; necesita el simulador para ejecutar MIPS.

La documentación anterior de TAC se conserva en docs/fase2.pdf. La implementación de esta fase y sus ejemplos se encuentran en el mismo repositorio, con las rutinas MIPS separadas del frontend para que pueda seguirse el recorrido desde cada cuádruplo hasta el código destino.